

Optimization for Machine Learning

Vishal Vikram Parth

September 19, 2026



Optimization Problem

$$\min_{x \in C} f(x), \quad C \subseteq \mathbb{R}^d, \quad f : \mathbb{R}^d \rightarrow \mathbb{R}$$

Object	Definition
Objective function	$f : \mathbb{R}^d \rightarrow \mathbb{R}$, scalar quantity to be minimized.
Decision variable	$x = (x_1, \dots, x_d)^T \in \mathbb{R}^d$.
Feasible set	$C = \{x : g_i(x) \leq 0, h_j(x) = 0\}$.
Optimal value	$f^* = \inf_{x \in C} f(x)$.
Solution set	$\arg \min_{x \in C} f(x) = \{x \in C : f(x) = f^*\}$.

Global and Local Optima

Point	Condition
Global minimum	$f(x^*) \leq f(x)$ for all $x \in C$
Local minimum	$\exists r > 0$ such that $f(x^*) \leq f(x)$ for $x \in C \cap B(x^*, r)$
Global maximum	$f(x^*) \geq f(x)$ for all $x \in C$
Local maximum	$\exists r > 0$ such that $f(x^*) \geq f(x)$ for $x \in C \cap B(x^*, r)$

$$B(x^*, r) = \{x : \|x - x^*\| < r\}.$$

Gradient and Hessian

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

Gradient

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix} \in \mathbb{R}^d$$

Hessian

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_d \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_d^2} \end{bmatrix} \in \mathbb{R}^{d \times d}$$

Second-Order Taylor Expansion

Taylor Expansion

For $f \in C^2$,

$$f(x + h) = f(x) + \nabla f(x)^T h + \frac{1}{2} h^T \nabla^2 f(x) h + o(\|h\|^2)$$

Critical Points and First-Order Condition

Critical / Stationary Point

$$\nabla f(x^*) = 0$$

A stationary point may be a minimum, maximum, or saddle point.

First-Order Necessary Condition (FONC)

If x^* is an interior local minimum and f is differentiable, then

$$\nabla f(x^*) = 0.$$

Local minimum $\implies$ Stationary point.

Stationary point $\nRightarrow$ Local minimum.

Second-Order Optimality Conditions

Condition	Condition at x^*	Role
FONC	$\nabla f(x^*) = 0$	Necessary
SONC	$\nabla f(x^*) = 0, \nabla^2 f(x^*) \succeq 0$	Necessary
SOSC	$\nabla f(x^*) = 0, \nabla^2 f(x^*) \succ 0$	Sufficient

Key distinction

SONC does not guarantee a minimum; SOSOC guarantees a **strict local minimum**.

Positive and Negative Definiteness

For a symmetric matrix A :

Type	Quadratic form	Eigenvalues
Positive definite	$v^T A v > 0, \forall v \neq 0$	$\lambda_i > 0$
Positive semidefinite	$v^T A v \geq 0, \forall v$	$\lambda_i \geq 0$
Negative definite	$v^T A v < 0, \forall v \neq 0$	$\lambda_i < 0$
Negative semidefinite	$v^T A v \leq 0, \forall v$	$\lambda_i \leq 0$
Indefinite	Both positive and negative values	Mixed signs

Classification of Stationary Points

Let

$$H = \nabla^2 f(x^*), \quad \nabla f(x^*) = 0.$$

Hessian	Spectrum	Point
$H \succ 0$	$\lambda_i > 0$	Strict local minimum
$H \succeq 0$	$\lambda_i \geq 0$	Inconclusive
$H \prec 0$	$\lambda_i < 0$	Strict local maximum
$H \preceq 0$	$\lambda_i \leq 0$	Inconclusive
H indefinite	Mixed signs	Saddle point

PD $\Rightarrow$ minimum,	ND $\Rightarrow$ maximum,	Indefinite $\Rightarrow$ saddle.
---------------------------	---------------------------	----------------------------------

Convex Functions

Let C be a convex set.

Definition

$f : C \rightarrow \mathbb{R}$ is convex if

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y)$$

for all $x, y \in C$ and $0 \leq t \leq 1$.

First-Order Characterization

For differentiable f ,

$$f(y) \geq f(x) + \nabla f(x)^T (y - x).$$

Second-Order Characterization

For $f \in C^2$,

$$f \text{ convex} \iff \nabla^2 f(x) \succeq 0 \quad \forall x.$$

Consequences of Convexity

Central Property

Every local minimum of a convex function is a global minimum.

For differentiable convex f ,

$$\nabla f(x^*) = 0 \iff x^* \text{ is a global minimizer.}$$

Property	Consequence
Convex	Local minimum is global
Strictly convex	At most one minimizer
Strongly convex	Unique minimizer (when attained)
Concave	$-f$ is convex
Affine	Both convex and concave

L -Smooth Function

f is L -smooth if its gradient is Lipschitz continuous:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y.$$

For $f \in C^2$,

$$\nabla^2 f(x) \preceq LI$$

or equivalently,

$$\lambda_{\max}(\nabla^2 f(x)) \leq L.$$

Strong Convexity

μ -Strong Convexity

f is μ -strongly convex if $\mu > 0$ and

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2}\|y - x\|^2.$$

For $f \in C^2$,

$$\nabla^2 f(x) \succeq \mu I \quad \text{or equivalently, } \lambda_{\min}(\nabla^2 f(x)) \geq \mu.$$

Property	Hessian condition
Convex	$\nabla^2 f \succeq 0$
L -smooth	$\nabla^2 f \preceq LI$
μ -strongly convex	$\nabla^2 f \succeq \mu I$
Both	$\mu I \preceq \nabla^2 f \preceq LI$

Gradient-Based Optimization

From Taylor Expansion to Descent

$$f(x + h) \approx f(x) + \nabla f(x)^T h$$

A direction h is a **descent direction** if

$$\nabla f(x)^T h < 0.$$

Among unit-norm directions, Cauchy–Schwarz gives the steepest-descent direction:

$$\arg \min_{\|h\|=1} \nabla f(x)^T h = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$$

Gradient Descent Update

Taking a step along the steepest-descent direction gives

$$x_{k+1} = x_k - \eta \nabla f(x_k), \quad \eta > 0.$$

Gradient Descent

Gradient Descent Update

Taking a step along the steepest-descent direction gives

$$x_{k+1} = x_k - \eta \nabla f(x_k), \quad \eta > 0.$$

Termination Criteria

Gradient descent can be stopped when any of the following conditions is satisfied:

$$\|\nabla f(x_k)\| \leq \varepsilon$$

$$|f(x_{k+1}) - f(x_k)| \leq \varepsilon$$

$$\|x_{k+1} - x_k\| \leq \varepsilon$$

$$k \geq k_{\max}$$

Generalized Descent

General Descent Scheme

$$x_{k+1} = x_k + \eta_k d_k$$

where d_k is a **descent direction** satisfying

$$\nabla f(x_k)^\top d_k < 0.$$

Some Choices

Method	Descent Direction d_k
Gradient Descent	$-\nabla f(x_k)$
Preconditioned / Adaptive	$-P_k \nabla f(x_k), \quad P_k \succ 0$
Newton	$-\nabla^2 f(x_k)^{-1} \nabla f(x_k)$

Choice of Hyperparameter

Choice of step size η :

$$0 < \eta < \frac{2}{L} \quad \text{for } L\text{-Lipschitz gradient.}$$

$$\text{or} \quad L = \lambda_{\max}(\nabla^2 f(x)) \quad \text{for Hessian-based } L.$$

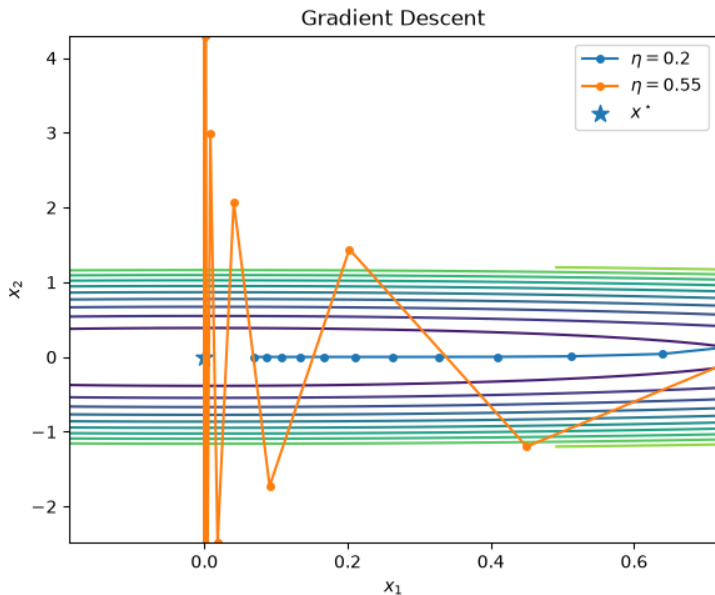
For diminishing step sizes, choose $\{\eta_k\}$ such that

$$0 < \eta_k, \quad \sum_{k=0}^{\infty} \eta_k = \infty, \quad \sum_{k=0}^{\infty} \eta_k^2 < \infty$$

A standard choice is

$$\eta_k = \frac{1}{k+1}$$

Gradient Descent



Stochastic Gradient Descent

Consider the empirical loss

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i(\theta).$$

Instead of computing the full gradient, sample a mini-batch B_t of size b :

$$g_t = \frac{1}{b} \sum_{i \in B_t} \nabla \ell_i(\theta_t)$$

and update

$$\theta_{t+1} = \theta_t - \eta_t g_t.$$

For uniformly sampled batches,

$$\mathbb{E}[g_t \mid \theta_t] = \nabla L(\theta_t), \quad \mathbb{E}\|g_t - \nabla L(\theta_t)\|^2 = \frac{\sigma^2}{b}.$$

Larger batch $\Rightarrow$ lower gradient variance

Example: SGD for Linear Regression

Consider the least-squares loss

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(y_i - x_i^\top \theta \right)^2.$$

For a mini-batch B_t of size b :

$$g_t = -\frac{1}{b} \sum_{i \in B_t} x_i \left(y_i - x_i^\top \theta_t \right).$$

The SGD update is

$$\theta_{t+1} = \theta_t - \eta g_t$$

Example: $\theta_t = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad B_t = \{i_1, i_2\}, \quad \eta = 0.01.$

Compute g_t using only the two observations and update θ_t .

Heavy-Ball Momentum

Gradient descent uses only the current gradient. Momentum incorporates past gradients:

$$v_t = \beta v_{t-1} + \nabla f(\theta_t), \quad 0 \leq \beta < 1, \quad v_{-1} = 0$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Equivalently,

$$v_t = \sum_{j=0}^t \beta^{t-j} \nabla f(\theta_j).$$

$$\text{Effective memory} \approx \frac{1}{1 - \beta} \text{ steps}$$

For example,

$$\beta = 0.9 \Rightarrow 10 \text{ steps}, \quad \beta = 0.99 \Rightarrow 100 \text{ steps}.$$

Nesterov Accelerated Gradient (NAG)

Instead of evaluating the gradient at the current point, NAG evaluates it at the **look-ahead point**:

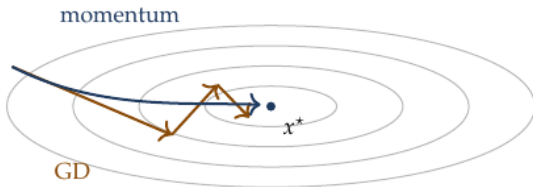
$$v_t = \beta v_{t-1} + \nabla f(\theta_t - \eta \beta v_{t-1})$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

$\underbrace{\theta_t - \eta \beta v_{t-1}}_{\text{look-ahead point}} \longrightarrow \text{evaluate gradient}$

The look-ahead gradient anticipates the effect of momentum and can reduce overshooting.

Momentum vs Gradient Descent



Gradient Descent (GD): follows the local gradient at each step, leading to a zig-zag trajectory in directions of high curvature.

Momentum: accumulates past gradients, damping oscillations and building velocity along the direction toward the minimizer x^* .

Adaptive Optimization: AdaGrad

Problem:

A single learning rate η is used for every parameter, although different coordinates may have very different gradient magnitudes.

$$\theta_{t+1} = \theta_t - \eta g_t$$

A large gradient in one coordinate can therefore lead to unnecessarily large updates, while a small-gradient coordinate may require larger steps.

Idea:

Adapt the learning rate separately for each coordinate using the history of its squared gradients.

AdaGrad

Let $g_t = \nabla L(\theta_t)$. Accumulate squared gradients coordinate-wise:

$$G_t = G_{t-1} + g_t^2 = \sum_{s=1}^t g_s^2, \quad G_0 = 0.$$

The update is

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \varepsilon} g_t$$

Hence, for coordinate j ,

$$\eta_{t,j} = \frac{\eta}{\sqrt{\sum_{s=1}^t g_{s,j}^2} + \varepsilon}.$$

AdaGrad: Advantages and Limitations

Advantages

- Coordinate-wise adaptive learning rates.
- Particularly effective for sparse gradients/features.
- Little tuning of per-coordinate scales.

Limitation

Since G_t is monotonically increasing,

$$\eta_{t,j} \longrightarrow 0.$$

Thus the effective learning rate can become too small, causing learning to stall on long or non-convex problems.

Adaptive Optimization: RMSProp

Problem:

AdaGrad accumulates *all* past squared gradients, so its effective learning rate keeps decreasing and may eventually become too small.

$$G_t = \sum_{s=1}^t g_s^2$$

Idea:

Instead of remembering all past gradients, use an exponentially weighted moving average of recent squared gradients.

Solution: RMSProp

Let $g_t = \nabla L(\theta_t)$. Maintain an exponentially weighted average:

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2, \quad 0 < \rho < 1.$$

The update is

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

Thus, the effective learning rate adapts to the recent gradient scale.

RMSProp: Advantages and Limitations

Advantages

- Adapts the learning rate coordinate-wise.
- Old gradients are gradually forgotten.
- Effective learning rate does not decay monotonically to zero.
- Adapts well to changing optimization landscapes.

Limitations

- Requires tuning η and ρ .
- Does not provide a general convergence guarantee.

Adaptive Optimization: Adam

Problem:

RMSProp adapts the learning rate using recent squared gradients, but it does not incorporate the directional information provided by momentum.

Idea: Combine momentum + adaptive scaling.

Solution: Adam

Let $g_t = \nabla L(\theta_t)$. Adam maintains estimates of the first and second moments:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad m_0 = 0, \quad \beta_1 \in (0, 1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad v_0 = 0, \quad \beta_2 \in (0, 1)$$

$$\mathbb{E}[m_t] = (1 - \beta_1^t) \mathbb{E}[g_t], \quad \mathbb{E}[v_t] = (1 - \beta_2^t) \mathbb{E}[g_t^2]$$

Bias-correct the moment estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The update is

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Adam: Advantages and Limitations

Advantages

- Combines momentum with coordinate-wise adaptive scaling.
- Robust to differences in gradient scale.
- Typically gives fast progress in the early stages of training.

Limitations

- Requires additional state: m_t and v_t .
- Requires tuning η , β_1 , and β_2 .
- Can converge to solutions with different generalization behavior.

Why Do We Regularize?

Overfitting:

A model may fit the training data extremely well by also fitting noise, leading to poor performance on unseen data.

Training loss $\ll$ Test loss $\implies$ overfitting

Regularization penalizes model complexity and encourages smaller parameters:

Larger $\lambda \implies$ smaller $\|\theta\| \implies$ lower variance

The resulting bias–variance trade-off is

Regularization $\implies$ higher bias, lower variance

Regularization: L2

Regularization adds a penalty on model complexity:

$$\min_{\theta} L(\theta) + \lambda R(\theta)$$

For **L2 regularization**,

$$R(\theta) = \frac{1}{2} \|\theta\|_2^2$$

Hence,

$$\min_{\theta} L(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

where $\lambda \geq 0$ controls the strength of regularization.

$$\nabla^2 \left(L(\theta) + \frac{\lambda}{2} \|\theta\|_2^2 \right) = \nabla^2 L(\theta) + \lambda I$$

Types of Regularization

$$\min_{\theta} L(\theta) + \lambda R(\theta)$$

Regularization	Penalty $R(\theta)$	Effect
L2 / Ridge	$\frac{1}{2} \ \theta\ _2^2$	Shrinkage
L1 / Lasso	$\ \theta\ _1$	Sparsity
Elastic Net	$\lambda_1 \ \theta\ _1 + \frac{\lambda_2}{2} \ \theta\ _2^2$	Sparsity + shrinkage

From Adam to Regularization

Problem:

Adam optimizes the training loss, but the resulting parameters may remain large, leading to an overly complex model and overfitting.

High model complexity $\implies$ Add a regularization term

$$\min_{\theta} L(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

Two common approaches are:

Adam + L2 regularization

or

AdamW

Adam + L2 vs AdamW

Adam + L2:

The regularisation term is added to the gradient:

$$g_t = \nabla L(\theta_t) + \lambda \theta_t$$

Then g_t is used in Adam's moment updates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

AdamW:

The moments use only the loss gradient:

$$g_t = \nabla L(\theta_t).$$

Weight decay is applied separately:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda \theta_t \right)$$

Learning-Rate Schedules

Instead of keeping η fixed, vary the learning rate with the iteration:

$$\eta_t = \text{learning rate at step } t$$

- **Constant:**

$$\eta_t = \eta_0$$

- **Step decay:**

$$\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}, \quad \gamma \approx 0.1$$

- **Exponential:**

$$\eta_t = \eta_0 e^{-ct}$$

- **Polynomial / $1/t$:**

$$\eta_t = \frac{\eta_0}{1 + ct}$$

- **Cosine:**

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right)$$

- **Warm-up + decay:** increase η_t initially, then apply a decay schedule.

Newton's Method: Derivation

For a twice-differentiable $f(x)$, approximate around x_k :

$$f(x_k + d) \approx f(x_k) + \nabla f(x_k)^\top d + \frac{1}{2} d^\top H_k d,$$

where

$$H_k = \nabla^2 f(x_k).$$

Minimize the quadratic approximation with respect to d :

$$\nabla f(x_k) + H_k d = 0.$$

Therefore, the Newton direction is

$$d_k = -H_k^{-1} \nabla f(x_k)$$

Newton Step Optimization

Using the Newton direction,

$$\boxed{x_{k+1} = x_k - H_k^{-1} \nabla f(x_k)} \quad H_k = \nabla^2 f(x_k).$$

Interpretation

$$\boxed{\nabla f(x_k) \rightarrow \text{local slope} \quad H_k \rightarrow \text{local curvature}}$$

Newton's method uses both slope and curvature to determine the step.

Application: Logistic Regression

For binary classification, let

$$P(y_i = 1 \mid x_i) = \sigma(x_i^T \beta), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The parameters β are obtained by minimizing the **negative log-likelihood**:

$$\min_{\beta} L(\beta) = - \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

where $p_i = \sigma(x_i^T \beta)$.

$$\nabla L(\beta) = X^T(p - y)$$

Hence gradient descent gives

$$\beta_{k+1} = \beta_k - \eta X^T(p - y)$$

Constrained Optimization

Unlike unconstrained optimization, the solution must satisfy prescribed **equality and inequality constraints**.

$$\begin{array}{ll} \min_{x \in \mathbb{R}^d} & f(x) \\ \text{s.t.} & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p. \end{array}$$

$$C = \{x : g_i(x) \leq 0 \ \forall i, \quad h_j(x) = 0 \ \forall j\} \quad (\text{feasible set})$$

- **Equality constraint** $h_j(x) = 0$: restricts x to a surface; always active.
- **Inequality constraint** $g_i(x) \leq 0$: defines a region; may be active or inactive.

$$g_i(x^*) = 0 \Rightarrow \text{active}, \quad g_i(x^*) < 0 \Rightarrow \text{inactive}.$$

Lagrange Multipliers: Equality Constraints

Consider

$$\min_{x \in \mathbb{R}^d} f(x) \quad \text{s.t.} \quad h(x) = 0.$$

Define the **Lagrangian**

$$\mathcal{L}(x, \lambda) = f(x) + \lambda h(x)$$

At a constrained optimum x^* ,

$$\nabla_x \mathcal{L}(x^*, \lambda^*) = \nabla f(x^*) + \lambda^* \nabla h(x^*) = 0$$

together with the constraint

$$h(x^*) = 0.$$

Hence,

$$\nabla f(x^*) = -\lambda^* \nabla h(x^*)$$

Geometric interpretation: $\nabla h(x^*)$ is normal to the constraint surface, so $\nabla f(x^*)$ must be parallel to it.

Karush–Kuhn–Tucker (KKT) Conditions

For

$$\min_x f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad h_j(x) = 0,$$

define the Lagrangian

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \mu_j h_j(x)$$

At a local optimum x^* , under a constraint qualification, there exist λ^*, μ^* satisfying the four KKT conditions:

1. Stationarity: $\nabla_x \mathcal{L}(x^*, \lambda^*, \mu^*) = 0$
2. Primal feasibility: $g_i(x^*) \leq 0, \quad h_j(x^*) = 0$
3. Dual feasibility: $\lambda_i^* \geq 0$
4. Complementary slackness: $\lambda_i^* g_i(x^*) = 0$

KKT Conditions: Interpretation

Condition	Meaning
Stationarity	No feasible descent direction remains
Primal feasibility	x^* satisfies all constraints
Dual feasibility	$\lambda_i^* \geq 0$ for inequality constraints
Complementary slackness	Inactive constraints have zero multiplier

$$\lambda_i^* g_i(x^*) = 0 \implies \begin{cases} g_i(x^*) < 0 \implies \lambda_i^* = 0, \\ g_i(x^*) = 0 \implies \lambda_i^* \geq 0. \end{cases}$$

For a convex problem

$$f, g_i \text{ convex}, \quad h_j \text{ affine},$$

KKT conditions are **sufficient for global optimality**.

Lagrangian Duality

Consider the constrained optimization problem

$$\begin{array}{ll} \min_x & f(x) \\ \text{s.t.} & g_i(x) \leq 0, \quad i = 1, \dots, m. \end{array}$$

Introduce multipliers $\lambda_i \geq 0$ and define the Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$$

The **dual function** is

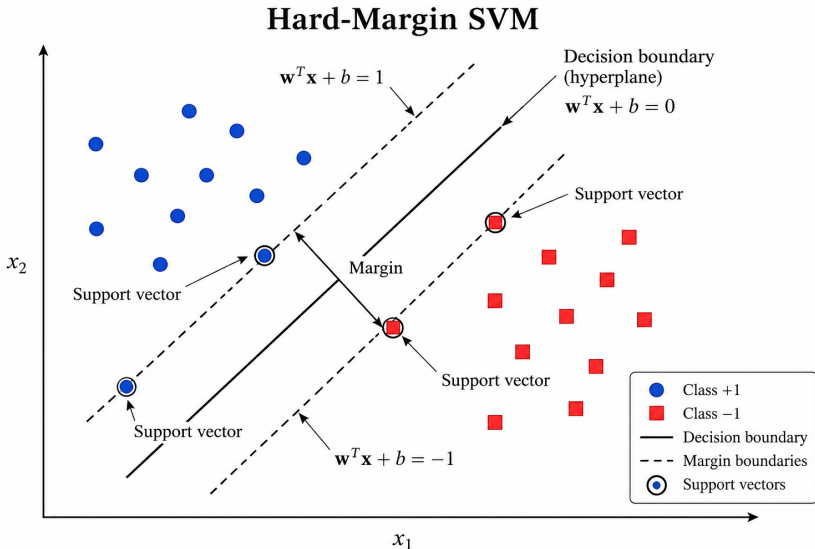
$$q(\lambda) = \inf_x \mathcal{L}(x, \lambda), \quad \lambda \geq 0.$$

The dual problem is therefore

$$\max_{\lambda \geq 0} q(\lambda).$$

Primal: optimize over x $\longleftrightarrow$ Dual: optimize over multipliers λ

Support Vector Machine



Support Vector Machine: The Problem

Given training data

$$\{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^d, \quad y_i \in \{-1, +1\},$$

consider a linear decision boundary

$$w^T x + b = 0.$$

The classifier is

$$\hat{y}(x) = \text{sign}(w^T x + b).$$

For a linearly separable dataset, many hyperplanes may classify all training points correctly.

SVM chooses the separating hyperplane with maximum margin.

The closest points to the decision boundary are called **support vectors**.

Maximum Margin Classifier

The two margin boundaries are normalized as

$$w^T x + b = 1, \quad w^T x + b = -1.$$

For every training point,

$$y_i(w^T x_i + b) \geq 1.$$

The distance between two parallel hyperplanes

$$w^T x + b = 1 \quad \text{and} \quad w^T x + b = -1$$

is

$$\frac{2}{\|w\|}.$$

Hence,

$$\text{maximize margin} \iff \text{minimize } \|w\|.$$

The points satisfying $y_i(w^T x_i + b) = 1$ are the support vectors.

Hard-Margin SVM: Primal Problem

For linearly separable data, maximize the margin subject to correct classification:

$$\max_{w,b} \frac{2}{\|w\|} \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1.$$

Equivalently, minimize a smooth objective:

$$\begin{array}{ll} \min_{w,b} & \frac{1}{2} \|w\|^2 \\ \text{s.t.} & y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n. \end{array}$$

This is a **convex quadratic program**:

$$\nabla^2 \left(\frac{1}{2} \|w\|^2 \right) = I \succeq 0,$$

and the constraints are affine.

Hard-Margin SVM: Lagrangian

Write the constraints as

$$1 - y_i(w^\top x_i + b) \leq 0.$$

Introduce multipliers $\alpha_i \geq 0$:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i \left[1 - y_i(w^\top x_i + b) \right].$$

Stationarity with respect to w :

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0 \implies w = \sum_{i=1}^n \alpha_i y_i x_i.$$

Stationarity with respect to b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0.$$

Hard-Margin SVM: Dual Problem

Substituting

$$w = \sum_i \alpha_i y_i x_i$$

into the Lagrangian eliminates w and b .

The dual problem becomes

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

After solving for α ,

$$w^* = \sum_i \alpha_i^* y_i x_i.$$

Thus the primal parameters (w, b) are recovered from the dual variables.

Why Use the Dual Problem?

The dual depends on the data through only

$$x_i^T x_j.$$

More importantly, complementary slackness gives

$$\alpha_i [1 - y_i(w^T x_i + b)] = 0.$$

Therefore,

$$\begin{cases} \alpha_i = 0, & y_i(w^T x_i + b) > 1, \\ \alpha_i > 0, & y_i(w^T x_i + b) = 1. \end{cases}$$

Hence only points on the margin contribute to

$$w = \sum_i \alpha_i y_i x_i.$$

These points are the support vectors.

The remaining training points can have $\alpha_i = 0$ and do not affect the separating hyperplane.

Limitation of the Hard Margin

Hard-margin SVM requires

$$y_i(w^T x_i + b) \geq 1 \quad \forall i.$$

A single outlier can force the separating hyperplane to move substantially in order to satisfy this constraint.

This can produce a relatively narrow margin.

Hard margin $\implies$ zero violations
high sensitivity to outliers

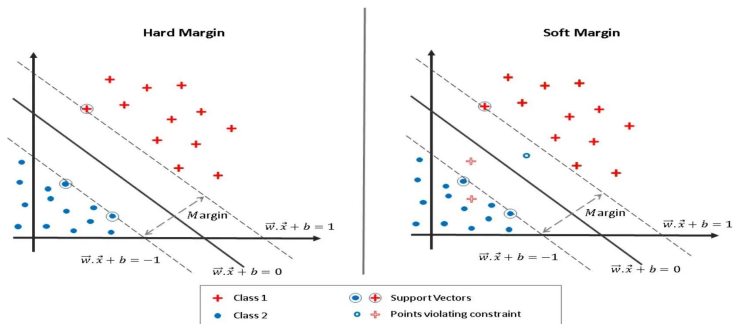
Instead, allow some points to violate the margin:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The variables ξ_i measure the amount of margin violation.

Hard-Margin vs Soft-Margin SVM

Hard Margin Vs Soft Margin



Soft-Margin SVM

Introduce slack variables $\xi_i \geq 0$:

$$y_i(w^T x_i + b) \geq 1 - \xi_i.$$

We now minimize both model complexity and violations:

$$\begin{array}{ll} \min_{w, b, \xi} & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} & y_i(w^T x_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0. \end{array}$$

Interpretation:

$\xi_i = 0$	outside/on margin
$0 < \xi_i < 1$	inside margin, correctly classified
$\xi_i > 1$	misclassified

C controls the trade-off between margin width and violations.

Soft-Margin SVM: Dual

The soft-margin dual is

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0. \end{aligned}$$

Compared with hard margin:

$$\alpha_i \geq 0 \quad \longrightarrow \quad 0 \leq \alpha_i \leq C.$$

Thus C limits the influence of individual points.

C large	few violations, narrower margin
C small	more violations, wider margin

Equivalently, eliminating ξ_i gives

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \underbrace{\max\{0, 1 - y_i f(x_i)\}}_{\text{Hinge Loss}}$$

Beyond Linear Separability: Features

A linear SVM cannot represent every useful decision boundary.

For example, suppose one class lies inside a disk while the other lies outside:

No line in $\mathbb{R}^2$ can separate the classes.

Instead, map the data into a higher-dimensional feature space:

$$\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d+1}.$$

For the parabolic lifting map,

$$\Phi(x) = \begin{bmatrix} x \\ \|x\|^2 \end{bmatrix}.$$

A spherical boundary in the original space becomes a **linear hyperplane** in the lifted space.

nonlinear boundary in $x \iff$ linear boundary in $\Phi(x)$

This is the geometric basis for the **kernel trick**.

From Nonlinear to Linear Separation

